

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# =====
# 1. Dataset Initialization & Overview
# =====
# Load the dataset (ensure the filename matches your downloaded file)
# The dataset contains 100,000 rows and 18 columns[span_3](start_span)[span_3](end_span).
file_path = 'retail_large_dataset.csv'
try:
    df = pd.read_csv(file_path)
    print("Dataset successfully loaded.")
except FileNotFoundError:
    print(f"Error: Please ensure '{file_path}' is in the current directory.")

# Display basic dataset information
print("\n--- Dataset Overview ---")
print(df.info())
print(df.head())

# Define numerical and categorical columns based on the assignment context
numerical_cols = ['product_price', 'final_price', 'discount_percentage', 'quantity', 'delivery_days', 'age']
categorical_cols = ['product_category', 'customer_segment', 'payment_method', 'return_status', 'shipping_type']

# =====
# 2. Univariate Analysis
# =====
print("\n--- Univariate Analysis: Numerical Variables ---")
# Compute Mean, Median, Standard Deviation, Min, Max, Skewness, and Kurtosis[span_4](start_span)[span_4](end_span)
univ_stats = pd.DataFrame()
for col in numerical_cols:
    if col in df.columns:
        univ_stats[col] = [
            df[col].mean(),
            df[col].median(),
            df[col].std(),
            df[col].min(),
            df[col].max(),
            df[col].skew(),
            df[col].kurtosis()
        ]
univ_stats.index = ['Mean', 'Median', 'Std Dev', 'Min', 'Max', 'Skewness', 'Kurtosis']
print(univ_stats.T)

print("\n--- Univariate Analysis: Categorical Variables ---")
# Analyze frequency distributions for categorical variables[span_5](start_span)[span_5](end_span)
for col in categorical_cols:
    if col in df.columns:
        print(f"\nFrequency Distribution for {col}:")
        print(df[col].value_counts(normalize=True) * 100) # Represented as percentages

# =====
# 3. Outlier Detection
# =====
print("\n--- Outlier Detection ---")
# Using Boxplots for visual detection[span_6](start_span)[span_6](end_span)
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
if 'product_price' in df.columns:
    sns.boxplot(y=df['product_price'], color='lightblue')
    plt.title('Boxplot: Product Price')

plt.subplot(1, 2, 2)
if 'final_price' in df.columns:
    sns.boxplot(y=df['final_price'], color='salmon')
    plt.title('Boxplot: Final Price')
plt.tight_layout()
plt.show()

# Interquartile Range (IQR) method for final_price[span_7](start_span)[span_7](end_span)
if 'final_price' in df.columns:
    Q1 = df['final_price'].quantile(0.25)
    Q3 = df['final_price'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers_iqr = df[(df['final_price'] < lower_bound) | (df['final_price'] > upper_bound)]
    print(f"Number of outliers in final_price (IQR method): {len(outliers_iqr)}")

# Z-score method for product_price[span_8](start_span)[span_8](end_span)
if 'product_price' in df.columns:
    z_scores = np.abs(stats.zscore(df['product_price'].dropna()))
    outliers_z = np.where(z_scores > 3)[0]
    print(f"Number of outliers in product_price (Z-score > 3): {len(outliers_z)}")

# =====

```

```

# 4. Bivariate Analysis
# =====
print("\n--- Bivariate Analysis ---")
# Numerical vs Numerical (Scatter Plots)
if all(col in df.columns for col in ['product_price', 'final_price']):
    plt.figure(figsize=(8, 5))
    sns.scatterplot(x='product_price', y='final_price', data=df, alpha=0.5)
    plt.title('Product Price vs Final Price')
    plt.show()

# Numerical vs Categorical (Boxplots)
if all(col in df.columns for col in ['customer_segment', 'final_price']):
    plt.figure(figsize=(10, 6))
    sns.boxplot(x='customer_segment', y='final_price', data=df)
    plt.title('Customer Segment vs Average Final Price')
    plt.show()

# =====
# 5. Correlation Matrix Analysis
# =====
print("\n--- Correlation Matrix ---")
# Perform correlation analysis using Pearson correlation[span_9](start_span)[span_9](end_span)
available_num_cols = [col for col in numerical_cols if col in df.columns]
if available_num_cols:
    corr_matrix = df[available_num_cols].corr(method='pearson')
    print(corr_matrix)

    # Generate correlation heatmap[span_10](start_span)[span_10](end_span)
    plt.figure(figsize=(10, 8))
    sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", vmin=-1, vmax=1)
    plt.title('Correlation Matrix of Numerical Variables')
    plt.show()

# =====
# 6. Multivariate Analysis
# =====
print("\n--- Multivariate Analysis ---")
# Examining interactions between three or more variables[span_11](start_span)[span_11](end_span)

# Example 1: Product Category x Discount x Final Price[span_12](start_span)[span_12](end_span)
if all(col in df.columns for col in ['product_category', 'discount_percentage', 'final_price']):
    # Create a discount bin for visualization
    df['discount_tier'] = pd.cut(df['discount_percentage'], bins=[-1, 10, 20, 100], labels=['Low', 'Moderate', 'High'])
    plt.figure(figsize=(12, 6))
    sns.barplot(x='product_category', y='final_price', hue='discount_tier', data=df, ci=None)
    plt.title('Final Price by Product Category and Discount Tier')
    plt.xticks(rotation=45)
    plt.show()

# Example 2: Customer Segment x Age x Spending[span_13](start_span)[span_13](end_span)
if all(col in df.columns for col in ['customer_segment', 'age', 'final_price']):
    df['age_group'] = pd.cut(df['age'], bins=[0, 30, 45, 100], labels=['<30', '30-45', '>45'])
    plt.figure(figsize=(10, 6))
    sns.barplot(x='customer_segment', y='final_price', hue='age_group', data=df, ci=None)
    plt.title('Spending by Customer Segment and Age Group')
    plt.show()

# =====
# 7. Skewness and Kurtosis Interpretation Check
# =====
print("\n--- Distribution Asymmetry Check ---")
# Plot distributions to visually confirm skewness and heavy tails (kurtosis)[span_14](start_span)[span_14](end_span)
if 'final_price' in df.columns:
    plt.figure(figsize=(8, 5))
    sns.histplot(df['final_price'], bins=50, kde=True, color='purple')
    plt.title('Distribution of Final Price (Checking for Right-Skew & Heavy Tails)')
    plt.show()

```